7. Interfaces and APIs

The Process Change Impact Simulator will provide a web-based interface developed using Python Flask, HTML, and CSS. Users will interact with the application through a web browser to create process models, define dependencies, test proposed changes, and review simulation results. Flask will connect the interface to the process analysis, simulation, and database components.

7.1 User Interface

- Dashboard: Displays saved processes, recent change scenarios, and links to the main functions.
- Process Builder: Allows process analysts to create processes, add activities, define conditions, and connect dependent activities.
- Change Scenario Page: Allows users to select an activity or rule and enter a proposed change.
- Impact Analysis Page: Displays downstream activities and dependencies affected by a selected change.
- Simulation Results Page: Compares original and modified process paths, skipped activities, newly required steps, and changed outputs.
- Report Page: Allows process managers to review and export analysis results.

7.2 Internal Application Interfaces

Flask routes will receive user requests and pass them to the appropriate Python modules.

Endpoint	Method	Purpose
/processes	GET	Display saved processes.
/processes/create	POST	Create and save a process.
/processes/<id>	GET	Display process details.
/processes/<id>/activities	POST	Add an activity to a process.
/processes/<id>/changes	POST	Save a proposed change scenario.
/changes/<id>/analyze	POST	Run dependency impact analysis.
/changes/<id>/simulate	POST	Compare original and modified processes.
/results/<id>	GET	Display simulation results.
/processes/import	POST	Import a process from a JSON file.
/processes/<id>/export	GET	Export a process definition as JSON.

Here, <id> represents the relevant process or change identifier. These endpoints are preliminary design choices and can be adjusted during implementation.

7.3 Data and Component Interfaces

- SQLite interface: Flask and its database layer will save and retrieve process records, dependencies, change scenarios, and simulation results.
- Graph interface: The NetworkX module will receive activity and dependency data and return graph analysis results.
- Simulation interface: The simulation engine will receive the original process and proposed changes and return execution paths and comparison results.

- Visualization interface: Plotly or Matplotlib will present dependency graphs and process comparisons.
- File interface: JSON import and export will allow users to transfer process definitions.

The initial system will use internal Python modules rather than external REST services. This design keeps the application simple and suitable for development and testing in VS Code.